

Empowering Compute, Network, Storage & AI

Tyrone S488 FRU Update Tool

Technical documentation, change log and user guide

Server / Host	Tyrone S488 / MDA200 (MSI S4251) (BMC 172.16.15.248)
Platform	AMD EPYC 9965 x2, 256 GiB, AMI MegaRAC BMC
Validation Tier	Tooling
Campaign Window	Validated 17 Jul 2026
Prepared For	Netweb / Tyrone Hardware Validation
Date	2026-07-17

Overall Result: PASS

1. Purpose

This document covers the FRU update tool shipped for the Tyrone S488 / MDA200 platform: why the original script did not work, what was changed, the features the tool now provides, and how to run it. The tool writes the BMC FRU inventory (chassis, board and product areas) over IPMI, either locally on the server or remotely against the BMC.

Everything described here was validated against live hardware (BMC 172.16.15.248, an S488 with an MSI S4251 mainboard). The evidence is in section 7.

2. The original problem

The supplied script `s488_tyrone_update_fru.sh` failed on the S488 with:

```
./s488_tyrone_update_fru.sh: line 104: 13074 Segmentation fault
ipmitool ... fru edit 0 field p 4 "MDA200A2G-48"
```

The script was not at fault - ipmitool itself crashed. The shell only reported that the program it called died on a signal.

Root cause

The FRU product area on this board was malformed: a field's type/length byte claimed more bytes than the 256-byte chip actually holds, and the area had no 0xC1 end marker. ipmitool's FRU parser walks off the end of its buffer and segfaults. It is the state of the chip that causes this, not the value being written - no string length avoids it.

The initial diagnosis (that the crash was caused by writing a string of a different length) was **disproved during live testing**: once the area was rebuilt cleanly, field edits that changed the string length worked normally. The corrected explanation is the one above.

The same defect made plain `ipmitool fru print` segfault, which matters: any tool that reads FRU by parsing `fru print` output will crash on these boards.

3. What was changed

#	Change	Why
1	Rewritten from bash to a single Python file	Portable; the image-editing logic (checksums, offsets) is not practical in shell. Old .sh + helper merged into one tool.
2	Whole-image write method added	Reads the FRU, edits it, rebuilds the area with correct length, checksum and end marker, writes it back with <code>fru write</code> . Never goes through ipmitool's crashing parser.
3	Automatic fallback (--method auto)	Tries the quick field edit first; if it crashes OR silently does not stick, the tool switches to the image method by itself.
4	Never calls 'fru print'	<code>fru print</code> segfaults on these boards. The tool reads the raw 'fru read' dump and decodes the binary itself.
5	Tolerant FRU decoder	Handles a missing end marker and a field whose length byte overruns the chip - keeps the readable bytes instead of discarding the field.
6	Extended to all FRU fields	Chassis, board and product areas. Requires rebuilding areas that sit before others and recomputing the common-header offsets + checksum.
7	Bogus MultiRecord offset repaired	ipmitool's field edit was seen writing a phantom MultiRecord offset into the header. The tool detects and clears it.
8	FRU device auto-detection (--auto-fru)	Finds the device holding a Product Serial instead of assuming id 0.
9	Safety: backup, confirm gate, read-back verify, audit log	See section 4.
10	Correct sudo handling	sudo is skipped on Windows and when already root; --no-sudo to force.

4. Features

Feature	Behaviour
Two write methods	field = one quick ipmitool call. image = read / edit / write the whole FRU. auto (default) tries field, falls back to image automatically.
Automatic crash recovery	Detects ipmitool dying on a signal (SIGSEGV) and falls back.
Silent-failure recovery	Detects a field edit that returns non-zero or simply does not take, by reading the value back, and falls back.
FRU auto-detection	--auto-fru scans devices and picks the one with a Product Serial.
Change preview + confirmation	Prints old -> new for every field and waits for 'y'. --yes skips it. Refuses to write unattended without --yes.
Automatic backup	Before every image write the current FRU is dumped to a timestamped .bin and the restore command is printed.
Read-back verification	After writing, values are re-read from the chip and compared.
Audit log	One line per run: time, host, FRU id, method, old -> new, RESULT.
Area repair	The image method rebuilds areas with correct length/checksum/end marker, fixing chips that crash ipmitool.
Space guard	If the new values do not fit the chip, the tool refuses and writes nothing rather than truncating data.
Safe password handling	The BMC password is passed to ipmitool via a 0600 temp file (-f), so it is not visible in the process list.

5. How to use the tool

5.1 Requirements

Python 3 and ipmitool. On the server both are normally present. To drive the BMC over the network from a workstation, both must be installed there - on Windows, WSL (Ubuntu) with `apt install ipmitool` works and is what was used for validation.

5.2 Basic use

Local (on the server itself):

```
sudo ./s488_tyrone_update_fru.py --auto-fru --productSN "8X4081110726"
```

Remote (against the BMC):

```
./s488_tyrone_update_fru.py --host 172.16.15.248 --user admin \
--pass '' --auto-fru --productSN "8X4081110726"
```

Before writing, the tool shows what it is about to do and waits:

```
Planned change on FRU device 0:
Product Serial      '8X4081110726' -> 'NEW-SERIAL-1'
```

```
Proceed with the write? [y/N]
```

Pass as many fields as you like in one run; anything not passed is left alone.

5.3 Fields you can set

Area	Flag	FRU field
Chassis	--chassisPN	Chassis Part Number
	--chassisSN	Chassis Serial Number
Board	--boardMfg	Board Manufacturer
	--boardProduct	Board Product Name
	--boardSN	Board Serial Number
	--boardPN	Board Part Number
	--boardFileId	Board FRU File ID
Product	--productMfg	Product Manufacturer
	--productName	Product Name
	--productPN	Product Part Number
	--productVersion	Product Version
	--productSN	Product Serial Number
	--assetTag	Product Asset Tag
	--productFileId	Product FRU File ID

5.4 Other options

Option	Meaning
--fru	FRU device id (default 0).
--auto-fru	Find the device holding a Product Serial. Recommended.
--method auto[field image]	auto (default) tries field then falls back to image. image forces the reliable whole-image path. field only tries the quick edit.
--host / --user / --pass	Remote BMC. All three together.
-y, --yes	Skip the confirmation prompt (for scripting).
--no-sudo	Do not prepend sudo in local mode.
--log / --no-log	Audit log path, or disable it.
--verb	Show every ipmitool call and each step.

5.5 Example - correcting product branding

```
./s488_tyrone_update_fru.py --host 172.16.15.248 --user admin \
  --pass '' --fru 0 \
  --productName "Tyrone Camarero" \
  --productPN "PDI200A2HG-810" --yes
```

If the quick field edit crashes or does not take, the tool reports it and completes the job through the image method without any further input.

6. Important notes

6.1 FRU changes only reach 'System Info' after a reboot

The BMC web UI shows two pages that look similar but are not the same data:

Page	Source	Updates when
FRU	The FRU chip - what this tool writes.	Immediately on write.
System Info	SMBIOS. The BIOS copies values out of the FRU during POST.	Only after the server reboots.

So after a FRU update the System Info page still shows the old values, or "To be filled by O.E.M." if read while the box is in BIOS setup or mid-boot. **This is not a failed write.** Reboot and POST picks the values up. This was confirmed on 172.16.15.248: after the FRU was corrected and the server rebooted, System Info changed on its own from "To be filled by O.E.M." to Tyrone Systems / Tyrone Camarero / 8X4081110726 / PDI200A2HG-810, with no DMI tool involved.

Exception: the Asset Tag on the System Info page is stored by the BMC, not derived from the FRU. Set it over Redfish:

```
curl -sk -u : -H 'Content-Type: application/json' \  
-H 'If-Match: *' -X PATCH \  
https://redfish/v1/Systems/Self \  
-d '{"AssetTag":"S18F260237"}'
```

Returns 204 on success. It returns 503 ("host may be in BIOS Setup") while the machine is booting - wait for the OS and retry.

6.2 The FRU chip is nearly full

The S488 FRU is 256 bytes and roughly 248 are already used - about 8 bytes spare. Replacing a value with one of similar length is fine, but giving a currently-empty field (e.g. Board Part Number) a real value may not fit. The tool refuses and writes nothing rather than truncating. Around 27 bytes can be reclaimed by trimming the trailing padding other fields carry.

7. Validation evidence

All of the following was exercised against the live board at BMC 172.16.15.248 (S488 / MSI S4251, dual EPYC 9965, 256 GiB) on 17 Jul 2026.

#	Check	Result
1	BMC reachable, IPMI authentication	PASS
2	ipmitool fru print on the malformed chip	SEGFAULT (defect confirmed)
3	Raw fru read dump + own decoder	PASS
4	--auto-fru locates the device holding the serial	PASS
5	Field edit crashes; tool auto-falls back to image	PASS
6	Field edit silently does not stick; caught by read-back	PASS
7	Image write of 9 fields across chassis+board+product	PASS
8	Read-back verification of every written field	PASS
9	Bogus MultiRecord header offset repaired (0x01 -> 0x00)	PASS
10	Backup taken before every image write	PASS
11	Oversized value refused, nothing written	PASS
12	FRU values appear in System Info after reboot	PASS
13	Redfish AssetTag PATCH	PASS (HTTP 204)

Not tested: local mode - every run above was remote via --host. Running the tool on the server itself (sudo, no --host) is expected to work but has not been executed. Only the S488 / FRU device 0 has been exercised.

7.1 Final state of 172.16.15.248

Chassis Part / Serial	PREGA2-812N / K61262059300154
Board Mfg / Product / Serial	Tyrone Systems / PE12XG / BR80S5B03600183
Product Mfg / Name	Tyrone Systems / Tyrone Camarero
Product Part Number	PDI200A2HG-810
Product Serial / Asset Tag	8X4081110726 / S18F260237
FRU header	0101 020b 1500 00dc - checksum valid, MultiRecord cleared
System Info (SMBIOS)	Tyrone Systems / Tyrone Camarero / 8X4081110726 / PDI200A2HG-810

No serial number was altered at any point. 8X4081110726, K61262059300154 and BR80S5B03600183 are the values the unit shipped with.

8. The fru_file reference images

The package ships one factory FRU image per board. They are reference copies - the serials inside are placeholders ("System serial number"), not real per-unit serials, so they must not be flashed as-is when a real serial is expected. They are useful for re-programming a blank chip, and for confirming how Tyrone fills each field in.

8.1 Tyrone field convention (from the factory images)

FRU field	Convention	Example
Product Manufacturer	Always Tyrone Systems	Tyrone Systems
Product Name	The descriptive name	Tyrone Camarero, PDB, Storage BP, MGX DC SCM
Product Part Number	The model code	TDA200A2R-44, MS-S405B
Product Version	The 609- / 709- part number	709-S488-01S

Server 172.16.15.248 shipped with Product Name and Product Part Number the wrong way round against this convention (a model code in the name field and the product name in the part-number field). They were swapped back to match.